

Cloud Computing Tools

Proyecto Final: Integración y Entrega de Tablero en la Nube

Pipeline multicloud serverless y orquestación con MicroK8s
para análisis geoespacial de expansión territorial en Chía

Juan Pablo Jiménez Harrizon Alonso Juan Fernando Serrano

Maestría en Analítica Aplicada — Universidad de La Sabana
Prof.: Hugo Franco, Ph.D.

15 de septiembre de 2026

Resumen. Se implementó de extremo a extremo un pipeline de datos *cloud-native* que ingiere información geoespacial de Foursquare Places mediante microservicios *Function as a Service* (AWS Lambda) programados con EventBridge Scheduler, la persiste en un Data Lake sobre Amazon S3 estructurado según la arquitectura *medallion* (*raw_zone* → *optimized_zone* → *consumption_zone*), la consolida en Amazon RDS PostgreSQL bajo un esquema *Database as a Service*, y expone la capa de visualización (Grafana) como un *Pod* contenerizado sobre un clúster Kubernetes ligero (MicroK8s) desplegado en una instancia EC2 *t3.medium*. El diseño aplica principios de *Zero Trust* en el control de acceso, idempotencia en todas las etapas de carga, y disciplina *FinOps* en la selección de servicios. El análisis resultante identifica tres estaciones de servicio en el corredor de La Caro como candidatas de expansión, frente a una saturación promedio de 21,5 competidores por radio de 500 m en el casco urbano.

Contenido

1	Formulación del proyecto	3
1.1	Contexto de negocio	3
1.2	Pregunta objetivo	3
1.3	Alcance	3
2	Arquitectura de la solución	3
2.1	Decisiones de diseño	3
2.2	Diagrama de la arquitectura implementada	4
2.3	Zero Trust en la implementación	6
3	Fase 1 — Ingesta serverless con rotación geográfica	6
3.1	La restricción de la fuente	6
3.2	Teselado rectangular como estrategia de superación del tope	6
3.3	Rotación del centro geográfico entre ejecuciones	7
3.4	Programación con EventBridge Scheduler	7
4	Fase 2 — Arquitectura medallion sobre S3	7
4.1	Deduplicación por identificador único	8
4.2	Justificación FinOps del formato columnar	8
5	Fase 3 — Persistencia transaccional en Amazon RDS	8
5.1	Cálculo del indicador de negocio	8
5.2	Idempotencia de la carga	9
5.3	Gestión de dependencias en el entorno FaaS	9

6 Fase 4 — Contenerización y orquestación	9
6.1 Contenerización con Docker sobre EC2 aprovisionado por código	9
6.2 Provisión del clúster MicroK8s	10
6.3 Namespaces y despliegue con Helm	10
6.4 Extracción y descifrado inverso de la credencial	11
6.5 Exposición del servicio	11
7 Verificación de la cadena automatizada	11
8 Hallazgo metodológico: sesgo de cobertura del muestreo	12
9 Capa de visualización: tablero en Grafana	13
9.1 Registro de RDS como origen de datos nativo	13
9.2 Composición del tablero	13
9.3 Decisiones de diseño de la visualización	14
10 Resultados y análisis de negocio	15
10.1 Indicadores obtenidos	15
10.2 Gradiente de saturación radial	15
10.3 Candidatas de expansión	15
10.4 Estrategia diferenciada por tipo de zona	15
11 Evaluación de escalabilidad y viabilidad futura	16
11.1 Limitaciones del clúster de nodo único	16
11.2 Alternativas evaluadas	17
11.3 Análisis FinOps	17
12 Reflexión crítica sobre la arquitectura	17
13 Conclusiones	18
13.1 Sobre la implementación técnica	18
13.2 Sobre el tablero en Grafana	19
13.3 Sobre la viabilidad futura	19
Anexo A. Estructura del repositorio	19
Anexo B. Glosario operativo	20

1 Formulación del proyecto

1.1 Contexto de negocio

Una cadena de restaurantes con operación en Chía (Cundinamarca) requiere priorizar zonas para su próxima sede. En las unidades previas del curso se exploró la densidad de restaurantes por categoría y la relación entre condiciones meteorológicas y afluencia. Queda por evaluar un tercer factor: la cercanía a nodos de alto tráfico vehicular. Las estaciones de servicio concentran paso constante de conductores y operan como polos de consumo asociado (comida rápida, cafeterías, tiendas de conveniencia) en su radio inmediato.

Foursquare Places expone la categoría *Gas Station / Garage* (4bf58dd8d48988d113951735) con los mismos atributos georreferenciados ya utilizados para restaurantes, lo que permite reutilizar el patrón de ingesta validado cambiando únicamente el parámetro de categoría.

La restricción operativa central es el AWS Academy Learner Lab: crédito limitado e irrecuperable. Toda decisión arquitectónica debe minimizar el tiempo de cómputo facturable y favorecer servicios que cobren por invocación en lugar de por hora encendida.

1.2 Pregunta objetivo

¿Qué estaciones de servicio del municipio de Chía presentan, dentro de un radio de 500 m, una baja densidad de restaurantes registrados en Foursquare —y por lo tanto constituyen nodos de alto tráfico vehicular con oferta gastronómica competidora insuficiente— y cómo debe la organización actuar de forma diferenciada sobre las zonas saturadas frente a las desatendidas?

La pregunta tiene dos componentes deliberados. El primero es descriptivo y geoespacial: localizar. El segundo es prescriptivo: definir qué hacer con cada tipo de zona, dado que una zona saturada no es simplemente “mala” —indica demanda comprobada— y una desatendida no es automáticamente “buena” —puede indicar ausencia de demanda.

1.3 Alcance

El estudio cubre el polígono administrativo de Chía delimitado por las coordenadas 4,825°–4,895° N y –74,095°––74,015° O, con un único radio de análisis de 500 m, elegido por corresponder a una distancia caminable típica desde un punto de reabastecimiento vehicular. La proximidad geográfica opera como variable *proxy* de tráfico: Foursquare no expone aforo vehicular, de modo que el resultado debe leerse como priorización exploratoria de zonas y no como estimación de demanda.

2 Arquitectura de la solución

2.1 Decisiones de diseño

La arquitectura responde a cuatro decisiones fundamentales, alineadas con el marco de *Architectural design decisions* trabajado en la Unidad 4.

Tabla 1: Racional conceptual de la arquitectura implementada.

Componente	Elección	Racional
Cómputo de ingesta	AWS Lambda (<i>FaaS</i>)	Dirigido por eventos, escala a cero, esfuerzo de mantenimiento nulo. Sustituye la instancia EC2 <code>t2.micro</code> de ingesta usada en unidades previas.
Almacenamiento	Amazon S3 (<i>Object Storage</i>)	Data Lake por zonas; el costo no depende de cómputo encendido. Formato columnar Parquet en las capas superiores.
Base de datos	Amazon RDS PostgreSQL (<i>DBaaS</i>)	Alta disponibilidad, parcheo y respaldos gestionados por el proveedor. Motor transaccional para el consumo del tablero.
Orquestación	MicroK8s sobre EC2 <code>t3.medium</code> (<i>IaaS</i>)	Plano de control Kubernetes completo con <i>self-healing</i> , sin el costo de gestión de un servicio administrado.
Identidad	Rol IAM (<code>LabRole</code>)	Credenciales temporales rotadas por el proveedor; acceso no humano sin secretos embebidos en código.
Red	Security Groups + <i>Namespaces</i>	Protección con estado a nivel de instancia y aislamiento lógico a nivel de clúster.

La tensión de fondo es *Infrastructure as a Service* frente a *Function as a Service*. IaaS ofrece control total a cambio de responsabilidad operativa y facturación por tiempo encendido; FaaS invierte ambos términos. El criterio aplicado fue *serverless primero*: toda carga de trabajo episódica —la ingesta, la transformación, la carga— se resolvió con Lambda, y IaaS quedó reservado exclusivamente para lo que exige un proceso residente: el clúster que sostiene el *Pod* de Grafana.

2.2 Diagrama de la arquitectura implementada

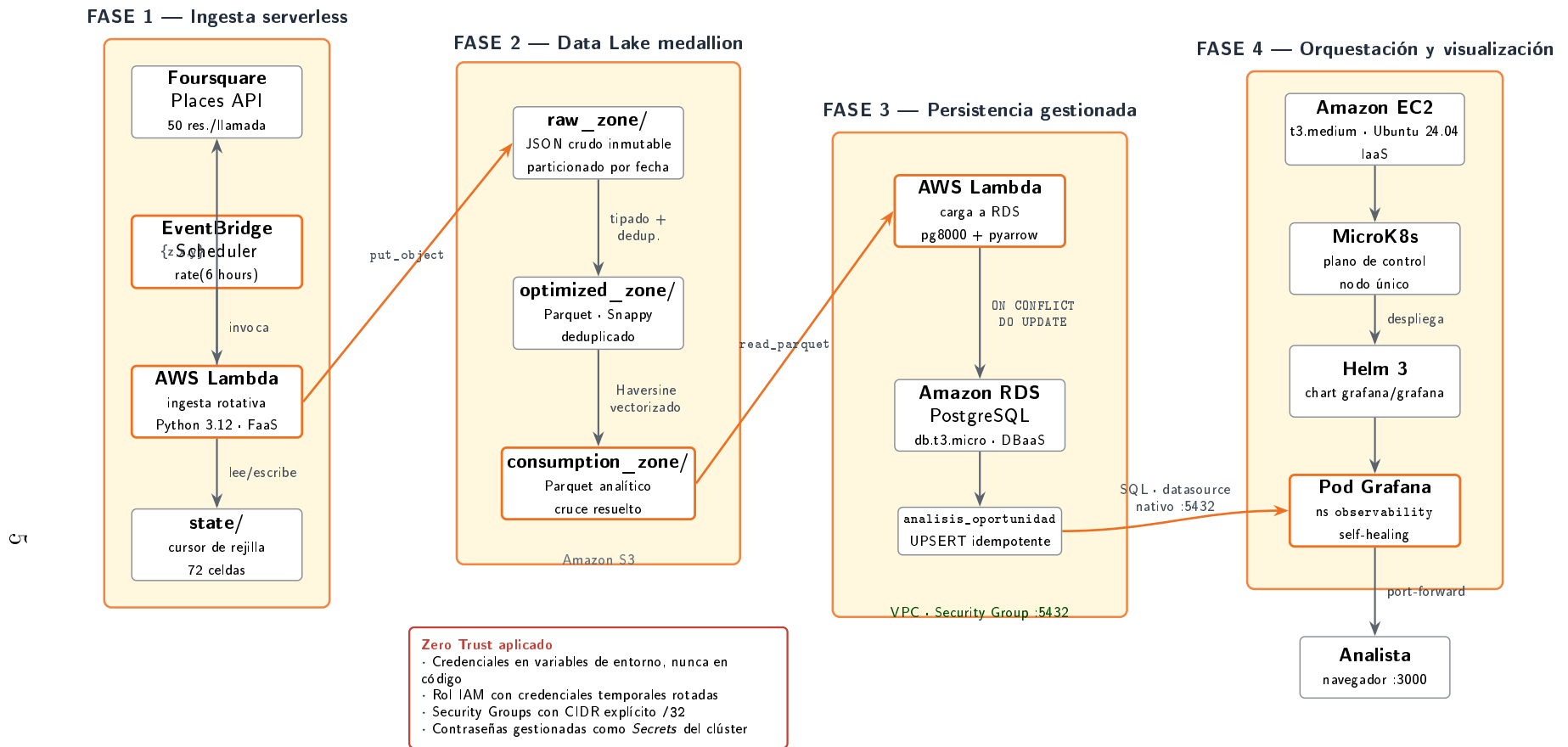


Figura 1: Arquitectura de la solución implementada. Las cuatro fases corresponden a los componentes construidos secuencialmente a lo largo del curso. Las flechas naranja marcan el flujo principal de datos; las azules, flujos internos de cada fase. Fuente: elaboración propia.

2.3 Zero Trust en la implementación

El principio “nunca confiar, siempre verificar” se materializó en cuatro controles concretos, no como declaración de intenciones:

- **Ningún secreto en el código.** Las claves de Foursquare y de RDS viven en variables de entorno de Lambda o en el archivo `.env` local, excluido del repositorio mediante `.gitignore`. El código fuente es publicable sin riesgo.
- **Identidad no humana con credenciales efímeras.** Las funciones Lambda acceden a S3 mediante el rol `LabRole`; AWS emite credenciales temporales en cada invocación y las rota automáticamente. No existe una llave de acceso permanente que pueda filtrarse.
- **Perímetro explícito.** El acceso a RDS se restringió a un CIDR `/32` correspondiente a la IP del clúster, en lugar de `0.0.0.0/0`. El puerto 22 se limitó a la IP del equipo de desarrollo.
- **Gestión delegada de contraseñas.** La credencial administrativa de Grafana nunca fue definida manualmente: Helm la generó aleatoriamente y Kubernetes la almacenó como objeto *Secret*, recuperable solo mediante autenticación contra la API del clúster.

3 Fase 1 — Ingesta serverless con rotación geográfica

3.1 La restricción de la fuente

El endpoint `/places/search` de Foursquare impone un tope duro de 50 resultados por llamada y **no ofrece paginación**. La revisión de la lista completa de parámetros aceptados —`query`, `ll`, `radius`, `fsq_category_ids`, `fsq_chain_ids`, `exclude_fsq_chain_ids`, `exclude_all_chains`, `fields`, `min_price`, `max_price`, `open_at`, `open_now`, `tel_format`, `ne`, `sw`, `near`, `sort`, `limit`— confirma que no existe `offset`, ni `cursor`, ni mecanismo para excluir identificadores ya obtenidos.

La consecuencia es que una consulta repetida al centro de Chía devuelve invariablemente los mismos 50 lugares más cercanos. El diagnóstico se validó empíricamente: el conjunto inicial de restaurantes contenía exactamente 50 registros, con distancia máxima al centro de 2.182 m —evidencia inequívoca de truncamiento por proximidad.

3.2 Teselado rectangular como estrategia de superación del tope

La solución implementada divide el municipio en una rejilla de celdas rectangulares y consulta una celda distinta en cada ejecución. Se emplean los parámetros `ne` y `sw` (esquinas noreste y suroeste) en lugar de `ll + radius`, porque los rectángulos embaldosan el plano sin dejar intersticios entre celdas vecinas, cosa que los círculos no logran sin solapamiento redundante.

Código 1: Construcción de la rejilla sobre el polígono de Chía.

```
BBOX_SUR, BBOX_OESTE = 4.825, -74.095
BBOX_NORTE, BBOX_ESTE = 4.895, -74.015
LADO_CELDA_M = 1000

def construir_rejilla():
    m_lat = 110574.0
    m_lon = 111320.0 * math.cos(math.radians(LAT_REF))
    paso_lat, paso_lon = LADO_CELDA_M / m_lat, LADO_CELDA_M / m_lon
    filas = ceil((BBOX_NORTE - BBOX_SUR) / paso_lat) # 8
    cols = ceil((BBOX_ESTE - BBOX_OESTE) / paso_lon) # 9
```

...

-> 72 celdas

Cada celda dispone de su propio cupo de 50 resultados, elevando el techo efectivo de 50 a $72 \times 50 = 3.600$ registros sobre un área de $7,7 \times 8,9$ km.

3.3 Rotación del centro geográfico entre ejecuciones

El estado de la rotación se persiste en S3 bajo el prefijo `state/`. Cada invocación lee el cursor, procesa un lote de celdas y avanza el índice:

Código 2: Avance del cursor y cierre de ciclo.

```
estado = leer_estado(bucket, perfil, len(celdas))
inicio = estado["proxima_celda"]
for k in range(lote):
    idx = (inicio + k) % len(celdas) # aritmetica modular: envuelve al final
    resultados = consultar_celda(celdas[idx], PERFILES[perfil], api_key)
    ...
siguiente = (inicio + procesadas) % len(celdas)
ciclo = estado["ciclo"] + (1 if siguiente <= inicio else 0)
```

Al completarse la rejilla el ciclo se incrementa y el barrido reinicia, refrescando los datos previamente capturados. Cada perfil —restaurantes y gasolineras— mantiene un cursor independiente (`state/restaurantes_cursor.json` y `state/gasolineras_cursor.json`), de modo que ambas ingestas avanzan sin interferirse.

3.4 Programación con EventBridge Scheduler

La ejecución recurrente se delegó a Amazon EventBridge Scheduler. Una sola función Lambda atiende ambos perfiles: el discriminante viaja en el *payload* del evento, no en el código.

Código 3: Resolución del perfil desde el evento.

```
perfil = (event or {}).get("perfil") or os.getenv("PERFIL", "restaurantes")
```

Tabla 2: Programaciones configuradas en EventBridge Scheduler.

Schedule	Expresión	Payload
ingesta_restaurantes	rate(6 hours)	{"perfil":"restaurantes"}
ingesta_gasolineras	rate(6 hours)	{"perfil":"gasolineras"}

Durante la implementación se detectó un defecto revelador: la primera programación se creó con *payload* vacío (`{}`), lo que hacía que todas las invocaciones cayeran al valor por defecto y procesaran únicamente restaurantes. El diagnóstico se resolvió por razonamiento sobre el código: las funciones `escribir_raw()` y `guardar_estado()` se ejecutan incondicionalmente, de modo que la ausencia total de artefactos bajo `raw_zone/gasolineras/` y `state/` probaba que la función jamás había sido invocada con ese perfil —no que hubiera fallado.

4 Fase 2 — Arquitectura medallion sobre S3

Las tres zonas responden a consumidores distintos, no a etapas decorativas.

Tabla 3: Zonas del Data Lake, patrón de escritura y propósito.

Zona	Formato	Patrón	Propósito
raw_zone/	JSON	Agrega	Fidelidad total a la respuesta original. Habilita reprocesamiento sin consumir cuota de la API.
optimized_zone/	Parquet	Lee-fusiona-sobrescribe	Deduplicado y tipado. Formato columnar comprimido para consulta analítica.
consumption_zone/	Parquet	Recalcula	Cruce espacial resuelto. Listo para carga sin cómputo adicional.

4.1 Deduplicación por identificador único

La capa optimizada fusiona lo recién ingerido con el histórico y reescribe el Parquet completo:

Código 4: Deduplicación en la zona optimizada.

```
df = pd.concat([df_previo, df_nuevo], ignore_index=True)
df = df.drop_duplicates(subset=["fsq_place_id"], keep="last")
```

La cláusula `keep="last"` implementa una política de refresco: si un establecimiento reaparece en un ciclo posterior con nombre o dirección actualizados, prevalece la versión más reciente.

Hallazgo de calidad de datos. La ingesta inicial empleaba el campo `name` como clave de deduplicación. La API devolvió 26 estaciones pero la tabla almacenó 24: dos pares de establecimientos comparten nombre de marca —*Terpel* y *Petrobras* en ubicaciones distintas— y colapsaron entre sí. La corrección consistió en adoptar `fsq_place_id`, identificador único por establecimiento que la ingesta original descartaba al seleccionar solo seis columnas de la respuesta.

4.2 Justificación FinOps del formato columnar

Parquet reduce el almacenamiento frente a CSV mediante compresión por columna (homogeneidad de tipo) y habilita *predicate pushdown* y *column pruning*: un motor de consulta serverless lee únicamente las columnas invocadas. Dado que servicios como Athena facturan por bytes escaneados, el formato columnar traduce directamente en reducción de costo por consulta.

Una decisión asociada merece registro. La zona de consumo publicaba inicialmente el resultado en Parquet y en CSV, este último por compatibilidad con el cargador existente. La duplicación contradecía el propio argumento FinOps que justifica el formato columnar —almacenamiento doble y dos copias susceptibles de desincronizarse— de modo que se eliminó el CSV y se adaptó el cargador para leer Parquet directamente.

5 Fase 3 — Persistencia transaccional en Amazon RDS

5.1 Cálculo del indicador de negocio

El cruce espacial se resuelve en la capa de consumo mediante la fórmula de Haversine vectorizada sobre NumPy, evitando la dependencia de la extensión PostGIS en la instancia RDS:

Código 5: Matriz de distancias gasolineras × restaurantes.

```
a = (np.sin((r_lat - g_lat) / 2) ** 2 +
```

```

    np.cos(g_lat) * np.cos(r_lat) * np.sin((r_lon - g_lon) / 2) ** 2)
dist = 2 * R * np.arcsin(np.sqrt(np.clip(a, 0, 1)))

competidores = (dist <= radio_m).sum(axis=1)
mas_cercano = dist.min(axis=1)

```

5.2 Idempotencia de la carga

La Lambda de carga implementa *upsert* contra la clave primaria, de modo que reejecutarla no duplica registros sino que refresca los existentes:

Código 6: Carga idempotente en RDS.

```

INSERT INTO analisis_oportunidad (fsq_place_id, gasolinera, ...)
VALUES (:fsq_place_id, :gasolinera, ...)
ON CONFLICT (fsq_place_id) DO UPDATE SET
    competidores_500m = EXCLUDED.competidores_500m,
    clasificacion     = EXCLUDED.clasificacion,
    actualizado_en    = NOW();

```

La propiedad es indispensable en un pipeline programado: conforme la ingesta rotativa descubre más restaurantes, los conteos cambian, y la carga debe poder ejecutarse repetidamente sobre la misma tabla sin efectos acumulativos indeseados.

5.3 Gestión de dependencias en el entorno FaaS

Lambda ejecuta sobre un micro-sistema operativo Linux, lo que genera incompatibilidades con librerías que incluyen binarios compilados para otra plataforma. Se aplicaron dos mecanismos complementarios:

- **Capa gestionada** para `pandas` y `pyarrow`: `arn:aws:lambda:us-east-1:336392948345:layer:AWSSDKPandas-P`. Evita empaquetar más de 200 MB de dependencias y consumir el límite del paquete de función.
- **Paquete de despliegue** para `pg8000`, construido en CloudShell con el indicador `--platform manylinux2014_x86_64`, que fuerza la descarga de binarios compatibles con el entorno de ejecución.

Capa y paquete coexisten: Lambda monta primero la capa en `/opt/python` y luego descomprime el paquete en `/var/task`; ambas rutas quedan en el `sys.path` del intérprete. La función de ingesta, por el contrario, no requiere empaquetado alguno: utiliza exclusivamente `urllib`, `json`, `math` y `boto3`, todos presentes en el *runtime*.

6 Fase 4 — Contenerización y orquestación

6.1 Contenerización con Docker sobre EC2 provisionado por código

El punto de partida fue el aprovisionamiento programático de una instancia EC2 mediante `boto3`, inyectando un bloque de *Infrastructure as Code* en el parámetro `UserData`, que `cloud-init` ejecuta una única vez en el primer arranque:

Código 7: Bloque `UserData` inyectado en el aprovisionamiento.

```

#!/bin/bash
dnf update -y          # actualizacion de repositorios
dnf install docker -y  # gestor de paquetes nativo

```

```
systemctl start docker
systemctl enable docker          # persistencia entre reinicios
usermod -aG docker ec2-user      # ejecucion sin privilegios de superusuario
docker run -d --name=grafana_server --restart unless-stopped \
    -p 3000:3000 grafana/grafana:latest
```

El mapeo `-p 3000:3000` vincula el puerto aislado del contenedor con el puerto expuesto del sistema anfitrión. Ese mapeo por sí solo no basta: el tráfico externo permanece bloqueado hasta que una regla de *Security Group* lo autorice explícitamente. La distinción entre aislamiento de red del contenedor y perímetro de la instancia es precisamente el control que Zero Trust exige hacer explícito.

6.2 Provisión del clúster MicroK8s

Sobre una instancia `t3.medium` (2 vCPU, 4 GiB) con Ubuntu 24.04 se instaló MicroK8s vía Snap. La instalación presentó un fallo reproducible que ilustra un riesgo real de la gestión de dependencias en producción:

Código 8: Fallo del canal por defecto.

```
fatal error: concurrent map read and map write
.../microk8s_plugins/sideload.go:81
snap.microk8s.daemon-containerd.service: Failed with result 'exit-code'.
```

El diagnóstico identificó una condición de carrera en un *plugin* propio de MicroK8s sobre containerd v2.1.6, presente en la versión del canal `latest/stable`. La mitigación consistió en fijar un canal anterior:

```
sudo snap install microk8s --classic --channel=1.32/stable
sudo usermod -a -G microk8s ubuntu
sudo snap alias microk8s.kubectl kubectl
```

La lección operativa es directa: en entornos productivos las versiones se fijan, no se toman del canal más reciente.

6.3 Namespaces y despliegue con Helm

El aislamiento lógico se implementó mediante un *Namespace* dedicado, y el despliegue mediante Helm, que actúa como gestor de plantillas y evita escribir manualmente los manifiestos YAML de *Deployment*, *Service* y *Secret*:

Código 9: Despliegue orquestado de la capa de visualización.

```
microk8s helm3 repo add grafana https://grafana.github.io/helm-charts
microk8s helm3 repo update
kubectl create namespace observability
microk8s helm3 install my-grafana grafana/grafana --namespace observability
kubectl get pods --namespace observability
```

Código 10: Verificación del Pod en ejecución.

NAME	READY	STATUS	RESTARTS	AGE
my-grafana-6698d56959-rrt2g	1/1	Running	1 (12m ago)	21h

La columna `RESTARTS` documenta el *self-healing* en operación: tras el apagado y reinicio de la instancia, el plano de control reconstruyó el *Pod* sin intervención manual. Es la propiedad que distingue un contenedor orquestado de uno ejecutado directamente con `docker run`.

6.4 Extracción y descifrado inverso de la credencial

Kubernetes protege las credenciales de aplicación como objetos *Secret*. Helm generó la contraseña administrativa de Grafana de forma aleatoria durante el despliegue —en ningún momento fue definida por el equipo— y el orquestador la almacenó codificada en base64. La recuperación requiere autenticación contra la API del clúster y decodificación explícita:

Código 11: Recuperación y descifrado inverso de la credencial.

```
kubect1 get secret -n observability my-grafana \  
-o jsonpath="{.data.admin-user}" | base64 -d; echo  
kubect1 get secret -n observability my-grafana \  
-o jsonpath="{.data.admin-password}" | base64 -d; echo
```

Conviene precisar la naturaleza del mecanismo: base64 es **codificación**, no cifrado. Su función no es proteger el valor en tránsito sino permitir que datos binarios arbitrarios se representen como texto dentro de un manifiesto YAML. La protección real proviene del control de acceso de la API de Kubernetes —quién puede ejecutar `kubect1 get secret`— y no de la codificación en sí. En un despliegue productivo esto se complementaría con cifrado en reposo del *etcd* y un gestor externo de secretos.

6.5 Exposición del servicio

El acceso se habilitó mediante `port-forward` dentro de una sesión `tmux` persistente:

```
tmux new -s grafana  
kubect1 port-forward --address 0.0.0.0 svc/my-grafana 3000:80 -n observability  
# Ctrl+b, d para desacoplar manteniendo el tunel activo
```

Se documenta explícitamente como limitación: `port-forward` es una herramienta de desarrollo y depuración. El túnel es un proceso que muere al cerrarse la sesión o al reiniciarse la instancia. Un entorno productivo emplearía un *Service* de tipo *NodePort* o *LoadBalancer*, mecanismos omitidos aquí por las restricciones presupuestales del Learner Lab.

7 Verificación de la cadena automatizada

Se ejecutaron secuencias de prueba sobre el encadenamiento completo, verificando la propagación ininterrumpida desde la fuente hasta la capa de visualización.

Tabla 4: Resultados de la verificación extremo a extremo.

Eslabón	Evidencia	Resultado
API → raw_zone	raw_zone/gasolineras/fecha=2026-09-12/*.json	JSON crudo particionado; una partición por corrida.
raw → optimized	optimized_zone/gasolineras/gasolineras.parquet	26 registros únicos tras deduplicación por fsq_place_id.
optimized consumption →	consumption/analisis_oportunidad.parquet	Cruce espacial resuelto; 26 filas con clasificación.
consumption → RDS	{"filas_procesadas": 26, "total_en_tabla": 26}	Carga idempotente confirmada por reejecución sin duplicados.
RDS → Grafana	Tablero con cuatro paneles activos	Consulta SQL directa sobre analisis_oportunidad.

Durante la verificación se detectó un defecto metodológico de primer orden, descrito en la sección siguiente, que obligó a rediseñar la estrategia de ingesta.

8 Hallazgo metodológico: sesgo de cobertura del muestreo

La primera ejecución del análisis clasificó 13 estaciones como “alta oportunidad”. La verificación cuantitativa desmontó el resultado:

- El conjunto de restaurantes contenía 50 registros con distancia máxima al centro de **2.182 m** — exactamente el tope de la API.
- Las estaciones de servicio se extendían hasta **4.521 m** del mismo punto.
- Las 13 estaciones marcadas como oportunidad se encontraban, **sin excepción**, más allá de los 2.182 m.

El conteo de cero competidores no evidenciaba ausencia de oferta gastronómica sino **ausencia de datos** en esa franja. La distancia al restaurante más próximo en esos casos —entre 1.332 y 3.025 m— confirmaba que el vecino más cercano pertenecía al casco urbano, no a la periferia.

La corrección fue doble. En el plano metodológico se incorporó a la tabla de consumo una columna `cobertura_valida`, cuyo umbral se recalcula en cada corrida a partir del alcance real del muestreo de restaurantes:

```
alcance_restaurantes = int(res["distance"].max())
out["cobertura_valida"] = out["dist_al_centro_m"] <= alcance_restaurantes
out["clasificacion"] = np.where(~out["cobertura_valida"], "Fuera de cobertura", ...)
```

En el plano operativo se implementó el teselado descrito en la Sección 3, que amplió la cobertura de restaurantes más allá de los 4.525 m. Tras la corrección, **las 26 estaciones quedaron dentro de cobertura válida** y el ranking pasó a ser interpretable en todo el municipio.

El principio general que se deriva es aplicable a cualquier análisis geoespacial comparativo: **los parámetros de muestreo de todas las fuentes deben homologarse antes de cruzarlas**, o el

resultado confunde ausencia de datos con ausencia de fenómeno.

9 Capa de visualización: tablero en Grafana

9.1 Registro de RDS como origen de datos nativo

Accediendo al servicio expuesto en el puerto 3000 y autenticando con la credencial recuperada del *Secret*, se registró Amazon RDS como origen de datos nativo con *host* apuntando al *endpoint* con sufixo :5432, base de datos *postgres* y modo TLS/SSL en *require*.

9.2 Composición del tablero

Tabla 5: Paneles del tablero y su función analítica.

Panel	Tipo	Función
Fila de indicadores	Stat ($\times 4$)	Cobertura del análisis, zonas de alta oportunidad, saturación promedio y validez del cruce. Responden las preguntas de primer orden.
Distribución territorial	Geomap	Las 26 estaciones georreferenciadas, coloreadas por clasificación. Revela el patrón espacial de forma inmediata.
Ranking de oportunidad	Table	Detalle ordenado por competencia ascendente. Soporta la pregunta “¿cuál exactamente?”.
Distribución completa	Bar chart	Las 26 estaciones ordenadas. Desactiva la objeción de selección conveniente de casos.

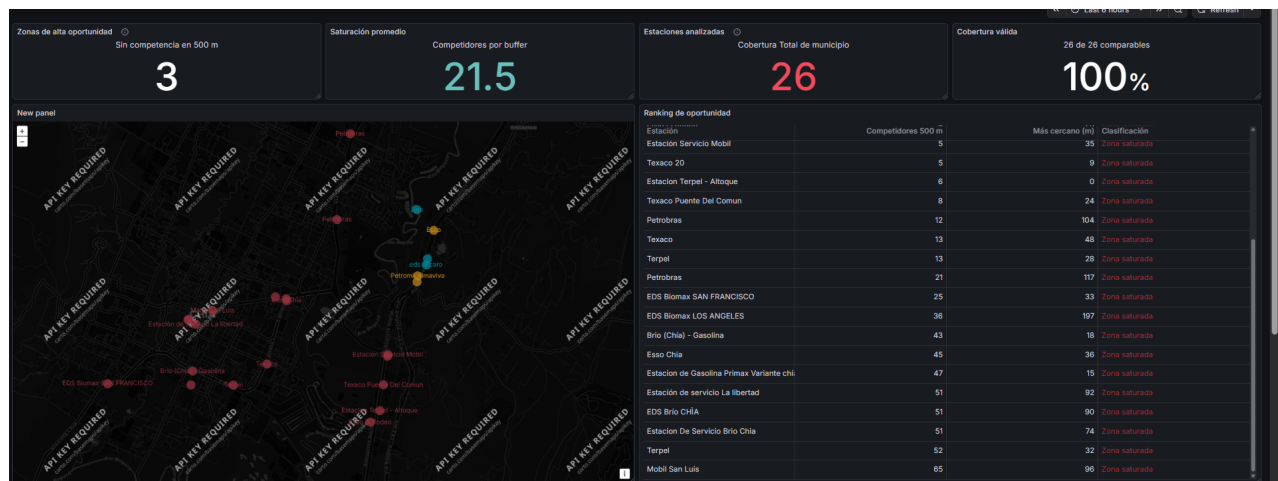


Figura 2: Tablero desplegado sobre el *Pod* de Grafana orquestado en MicroK8s. La fila superior presenta los cuatro indicadores agregados; el panel izquierdo georreferencia las 26 estaciones sobre cartografía de Chía, coloreadas por clasificación; el panel derecho despliega el ranking ordenado por competencia ascendente. Consultado directamente sobre la tabla *analisis_oportunidad* en Amazon RDS.

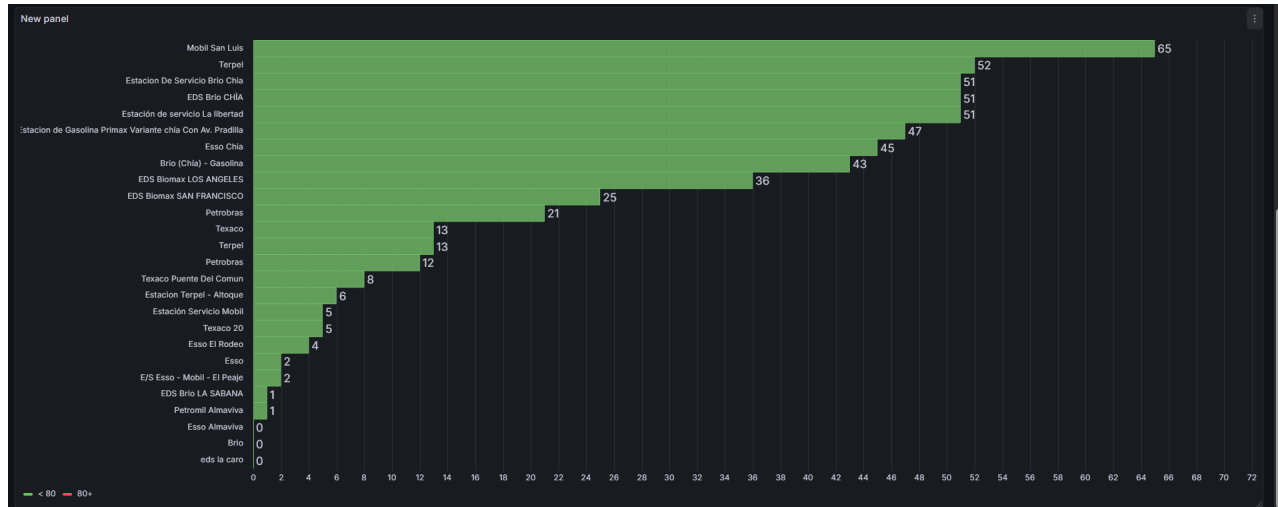


Figura 3: Distribución completa de competencia gastronómica por estación de servicio. El gradiente va de 65 competidores en Mobil San Luis, en pleno casco urbano, hasta las tres estaciones del corredor de La Caro sin competencia registrada en 500 m. La monotonía de la curva evidencia el patrón de saturación radial descrito en la Sección 10.2.

Código 12: Consulta del panel Geomap.

```
SELECT gasolinera, latitude, longitude, competidores_500m, clasificacion
FROM analisis_oportunidad;
```

El panel Geomap se configuró con vista inicial en coordenadas (4,8588; -74,0587) y *zoom* 12, capa de marcadores sobre campos *latitude/longitude*, y color determinado por el campo *clasificacion* mediante *value mappings* que fijan un color por valor de texto. Sin ese anclaje, Grafana asigna colores por orden de llegada de las series y la semántica cromática se invierte entre refrescos.

9.3 Decisiones de diseño de la visualización

Dos decisiones merecen justificación explícita:

Codificación cromática segura para daltonismo. Se descartó el esquema semáforo (verde-amarillo-rojo) porque el par verde/rojo es precisamente el que no distinguen las deficiencias de visión cromática más frecuentes, que afectan aproximadamente al 8 % de la población masculina. Se adoptó una escala divergente teal-ocre-granate, validada mediante simulación de protanopia, deuteranopia y tritanopia.

Precálculo en la capa de consumo. Las consultas del tablero son lecturas directas de una tabla, sin *CROSS JOIN* ni cálculo de distancias. El cruce espacial se resuelve una vez en la capa de consumo y no en cada refresco del panel. Esto reduce carga sobre la instancia RDS y desacopla el costo del tablero del número de usuarios que lo consultan.

10 Resultados y análisis de negocio

10.1 Indicadores obtenidos

Tabla 6: Indicadores del análisis sobre el municipio de Chía.

Indicador	Valor	Lectura
Estaciones de servicio analizadas	26	Cobertura total del municipio.
Cobertura válida del cruce	100 %	Las 26 dentro del alcance del muestreo.
Zonas de alta oportunidad	3	Sin competencia registrada en 500 m.
Zonas de oportunidad media	4	Entre 1 y 2 competidores.
Zonas saturadas	19	Tres o más competidores.
Saturación promedio del mercado	21,5	Restaurantes por radio de 500 m.
Saturación máxima observada	65	Mobil San Luis, casco urbano.

10.2 Gradiente de saturación radial

Los datos revelan un patrón monótono: la saturación competitiva decrece con la distancia al centro. Las estaciones más céntricas —La Libertad, EDS Brío Chía, E/S Brío Chía, todas a menos de 470 m del centro— concentran 51 competidores cada una, y Mobil San Luis alcanza 65. Las ubicadas en el corredor nororiental, a más de 4 km, no registran ninguno. El patrón confirma la estructura urbana esperada y, al haberse corregido el sesgo de cobertura, ahora es interpretable como fenómeno real.

10.3 Candidatas de expansión

Tabla 7: Estaciones clasificadas como alta oportunidad.

Estación	Competidores 500 m	Más cercano (m)	Dist. centro (m)
■ eds la caro	0	739	4.173
■ Esso Almaviva	0	737	4.227
■ Brio	0	549	4.419

Las tres se ubican en el corredor de La Caro sobre la Autopista Norte, con el restaurante más próximo a más de medio kilómetro.

10.4 Estrategia diferenciada por tipo de zona

El hallazgo central del análisis de negocio es que **la clasificación no es un ordenamiento de bueno a malo**, sino un mapa de estrategias distintas. Una lectura ingenua del ranking recomendaría abrir en las tres de cero competencia y descartar el resto; el análisis correcto es más matizado.

Zonas saturadas (19 estaciones, 3–65 competidores). La densidad de restaurantes es evidencia de demanda comprobada, no solo de competencia. Que 51 establecimientos coexistan alrededor de la estación La Libertad indica que el mercado sostiene esa oferta. La implicación estratégica no es evitar la zona sino que **la entrada exige diferenciación, no replicación**: un formato idéntico al de los competidores instalados compite por participación en un mercado ya repartido, con alto costo de adquisición de cliente. Las opciones viables son un nicho no cubierto por la oferta existente, un formato de servicio distinto (entrega, ventanilla vehicular, horario extendido), o una decisión de

no entrar y consolidar la operación actual. Adicionalmente, esas zonas funcionan como *benchmark*: permiten estimar qué densidad de oferta tolera el mercado local antes de saturarse.

Zonas de alta oportunidad (3 estaciones, 0 competidores). La ausencia de competencia admite dos interpretaciones opuestas, y distinguirlas es la diferencia entre una decisión informada y una apuesta. Puede indicar **mercado desatendido** —tráfico sin oferta, la hipótesis favorable— o **mercado inviable** —nadie ha abierto allí porque el flujo no sostiene un establecimiento—. Los datos disponibles no permiten decidir entre ambas, y el informe no debe pretender lo contrario. La recomendación es tratarlas como **candidatas a validación, no a inversión**: antes de comprometer capital, levantar aforo vehicular real, verificar disponibilidad y costo de predios, y confirmar accesibilidad desde la vía. La ventaja de la posición es real —captura del total del tráfico sin fragmentación— pero está condicionada a que ese tráfico exista en volumen suficiente.

Zonas de oportunidad media (4 estaciones, 1–2 competidores). Son las candidatas más equilibradas desde la gestión de riesgo. La presencia de uno o dos establecimientos valida que el mercado sostiene operación comercial —lo que la zona de cero competidores no demuestra— sin implicar la saturación del casco urbano. Un caso ilustra el matiz: E/S Esso-Mobil El Peaje registra 2 competidores, pero el más cercano está a 71 m; la competencia es directa e inmediata. En contraste, Petromil Almagro tiene 1 competidor a 489 m, una distancia que en la práctica no compite por el mismo cliente de paso. **El conteo por sí solo es insuficiente: la distancia al competidor más cercano desempata.**

Recomendación operativa. Priorizar las zonas de oportunidad media para expansión en el corto plazo, someter las tres de alta oportunidad a validación de campo antes de decidir, y utilizar las zonas saturadas como referencia de mercado y como candidatas a formatos diferenciados si la cadena decide competir en el casco urbano.

11 Evaluación de escalabilidad y viabilidad futura

11.1 Limitaciones del clúster de nodo único

El clúster implementado ofrece *self-healing* pero **no alta disponibilidad**. La distinción es sustantiva y frecuentemente se confunde:

- **Self-healing (presente).** Si el proceso del contenedor falla, el plano de control reconstruye el *Pod*. La columna **RESTARTS** del despliegue lo evidencia.
- **Alta disponibilidad (ausente).** Si falla el nodo —la instancia EC2, la zona de disponibilidad, el plano de control mismo— no existe otro nodo al cual reprogramar la carga. El servicio se pierde por completo.

Bajo estrés productivo, las restricciones concretas son: techo de escalamiento limitado a los recursos de una **t3.medium** (2 vCPU, 4 GiB); imposibilidad de actualización sin interrupción, dado que no hay nodo alterno donde drenar la carga; punto único de fallo en el plano de control; y exposición mediante **port-forward**, que no sobrevive reinicios ni distribuye carga.

11.2 Alternativas evaluadas

Tabla 8: Comparación de estrategias de despliegue y sus compensaciones FinOps.

Estrategia	Ventaja	Costo	Cuándo se justifica
MicroK8s nodo único (<i>implementada</i>)	Costo mínimo; plano de control completo; control total de la configuración.	Sin alta disponibilidad; operación manual; techo de un solo nodo.	Desarrollo, pruebas, cargas internas tolerantes a interrupción.
Kubernetes multinodo con LoadBalancer	Tolerancia a fallo de nodo; actualizaciones sin interrupción; distribución de carga.	Tres o más instancias encendidas; balanceador facturado por hora; complejidad de red.	Servicios con compromiso de disponibilidad hacia terceros.
Amazon EKS (<i>managed</i>)	Plano de control gestionado y redundante; integración nativa con IAM y balanceadores.	Tarifa fija del plano de control más el costo de los nodos trabajadores.	Organizaciones con múltiples equipos y varias cargas contenerizadas.
Grafana Cloud (<i>SaaS</i>)	Cero infraestructura propia; escalamiento y respaldo delegados.	Suscripción por usuario; dependencia de proveedor; dato sale del perímetro.	Cuando la visualización es el único componente que requiere alta disponibilidad.

11.3 Análisis FinOps

El eje de la decisión no es el costo de cómputo sino **qué se paga por no gestionar**. Un plano de control gestionado cuesta más por hora que uno autogestionado, pero elimina horas de ingeniería dedicadas a parcheo, respaldo y recuperación —horas que en una organización tienen un costo superior al del cómputo evitado.

La disciplina aplicada en este proyecto fue coherente con la restricción presupuestal: cómputo episódico delegado a FaaS, que factura por milisegundo de ejecución; instancias EC2 y RDS detenidas al terminar cada sesión de trabajo; almacenamiento columnar comprimido en las capas superiores del lago; y ausencia deliberada de balanceadores, que facturan por hora encendida con independencia del tráfico.

Para esta carga de trabajo específica —dos conjuntos de datos que suman menos de un megabyte y un tablero interno— la recomendación honesta es que **ninguna de las alternativas de alta disponibilidad se justifica económicamente**. El escalamiento a multinodo o a EKS solo adquiere sentido cuando la indisponibilidad tiene un costo medible para la organización, condición que no se cumple aquí.

12 Reflexión crítica sobre la arquitectura

Esta sección se incluye deliberadamente porque la honestidad analítica sobre las propias decisiones es parte del criterio profesional que el ejercicio busca desarrollar.

En el plano académico, la redundancia de la arquitectura es su justificación. El pipeline recorre intencionalmente cada modelo de servicio de la nube: IaaS en el aprovisionamiento de EC2, FaaS en las funciones Lambda, DBaaS en RDS, almacenamiento de objetos en S3, y orquestación

de contenedores en Kubernetes. Cada capa existe para que sea ejercitada, comprendida y evaluada. Bajo ese objetivo, la arquitectura cumple.

En el plano profesional, el flujo implementado no sería eficiente. Vale la pena nombrar las ineficiencias con precisión, porque reconocerlas es más valioso que defenderlas:

- **Redundancia de persistencia.** Los mismos datos residen en S3 (tres veces, una por zona) y en RDS. En producción se elegiría uno: Athena consultando Parquet directamente —sin base de datos— o PostgreSQL sin lago. Mantener ambos duplica el almacenamiento y crea dos fuentes que pueden desincronizarse.
- **Medallion sobredimensionado.** Tres zonas con transformación entre capas para un conjunto de 26 y algunos cientos de registros. El beneficio del formato columnar y de la separación por zonas aparece con volúmenes de varios órdenes de magnitud superiores, múltiples equipos consumidores, o necesidades frecuentes de reprocesamiento. A esta escala, el beneficio es formativo, no económico.
- **Orquestación sin nada que orquestar.** Kubernetes gestiona un solo *Pod* en un solo nodo. El valor de un orquestador —programación entre nodos, escalamiento horizontal, despliegue progresivo, descubrimiento de servicios— requiere una flota para manifestarse. Aquí se pagó la complejidad completa del plano de control por una única propiedad utilizable: el reinicio automático, que `docker run --restart unless-stopped` ya proporcionaba en el taller anterior.
- **Costo real mal ubicado.** El gasto dominante de esta solución no es el cómputo —son céntimos— sino las horas de ingeniería invertidas en construirla y la superficie operativa que deja: dos funciones Lambda, dos programaciones, tres zonas de almacenamiento, una base de datos, un clúster y un túnel manual. Cada pieza es algo que puede romperse y que alguien debe mantener.

Qué haría un equipo profesional con este problema. Un script en Python de aproximadamente cien líneas, ejecutado por una función programada, escribiendo directamente en PostgreSQL, con el tablero servido por Grafana Cloud en su nivel gratuito. Tiempo de construcción: una tarde. Superficie operativa: dos componentes. Costo mensual: prácticamente cero.

La conclusión no invalida el ejercicio —lo contextualiza. El valor de haber construido la versión completa es que ahora el equipo puede **justificar por qué no la construiría así**, y esa capacidad de discriminar cuándo una arquitectura se paga sola y cuándo es sobreingeniería constituye precisamente el criterio que distingue a un arquitecto de nube de un implementador de tutoriales.

13 Conclusiones

13.1 Sobre la implementación técnica

Se verificó la cadena automatizada de extremo a extremo: ingesta dirigida por eventos desde una API externa hacia un Data Lake en zonas, transformación serverless hacia formato columnar, carga idempotente en un motor transaccional gestionado, y consumo desde una aplicación contenerizada bajo orquestación Kubernetes. Cada eslabón fue probado de forma independiente y en conjunto.

La restricción de 50 resultados por llamada de la API —sin paginación ni exclusión por identificador— se superó mediante teselado rectangular del municipio con rotación de celda entre ejecuciones, elevando el techo efectivo de 50 a 3.600 registros. El estado de la rotación se persiste en el propio Data Lake, lo que hace la solución **sin estado** a nivel de función: cualquier invocación puede reconstruir su contexto desde S3.

13.2 Sobre el tablero en Grafana

El tablero cumple su función: traduce un cruce espacial de 26 estaciones contra varios cientos de restaurantes en cuatro paneles que responden preguntas de negocio, no que muestran datos. Tres conclusiones se desprenden de su construcción.

Primero, **el precálculo en la capa de consumo fue la decisión correcta**. Las consultas del tablero son lecturas planas; el cruce espacial no se recalcula en cada refresco. Esto mantiene la carga sobre RDS constante con independencia del número de usuarios, y es lo que justifica arquitectónicamente la existencia de la capa de consumo.

Segundo, **el panel Geomap es el que transporta el hallazgo**. La tabla y el gráfico de barras son evidencia de soporte, pero es el mapa el que comunica de forma inmediata que la oportunidad se concentra en el corredor nororiental y la saturación en el casco urbano. Al presentar ante una audiencia directiva, el mapa es el panel que sostiene la narrativa.

Tercero, **el tablero expone honestamente sus propios límites**. La columna `cobertura_valida` y la categoría “Fuera de cobertura” son un mecanismo deliberado para impedir que una ausencia de datos se lea como ausencia de mercado. Un tablero que oculta sus zonas ciegas es más peligroso que uno que no existe, porque produce decisiones con confianza injustificada.

13.3 Sobre la viabilidad futura

El clúster de nodo único es adecuado para el propósito actual y explícitamente insuficiente para un compromiso de disponibilidad hacia terceros. La ruta de evolución —multinodo con balanceador, EKS gestionado, o visualización como servicio— depende de una variable que este proyecto no tiene: el costo organizacional de la indisponibilidad. Mientras ese costo sea despreciable, escalar la arquitectura es gasto sin retorno.

Anexo A. Estructura del repositorio

```
proyecto_final/
|-- README.md                Documentacion y decisiones tecnicas
|-- .gitignore               Excluye .env, *.pem, credentials.json
|-- requirements.txt
|-- infraestructura/
|   |-- docker_deployment.py  Aprovisionamiento EC2 + UserData (IaC)
|   |-- diagnostico_ec2.py    Verificacion de estado y Security Groups
|-- ingesta/
|   |-- lambda_medallion.py    Ingesta rotativa - tres capas medallion
|   |-- gasolineras_pipeline.py Ejecucion local Foursquare -> RDS
|   |-- probar_lambda_local.py Invocacion local del handler
|-- carga/
|   |-- lambda_carga_rds.py     consumption (Parquet) -> RDS - UPSERT
|-- sql/
|   |-- 01_crear_tabla.sql
|   |-- 02_analisis_oportunidad.sql
|-- dashboard/               Capturas del tablero
'-- informe/                 Documento LaTeX y PDF
```

Anexo B. Glosario operativo

Término	Definición aplicada en este proyecto
Idempotencia	Propiedad por la cual ejecutar una operación repetidamente produce el mismo estado final. Implementada mediante <code>ON CONFLICT DO UPDATE</code> en la carga a RDS y <code>drop_duplicates</code> en la capa optimizada.
Zero Trust	Modelo que no asume confianza por ubicación de red. Materializado en credenciales efímeras por rol, CIDR explícitos y secretos gestionados por el orquestador.
Deduplicación	Eliminación de registros repetidos por clave única (<code>fsq_place_id</code>), no por nombre, evitando el colapso de establecimientos homónimos.
FinOps	Disciplina de gestión financiera del gasto en nube. Aplicada mediante preferencia por facturación por invocación, apagado de instancias y formato columnar.
IaaS	<i>Infrastructure as a Service</i> . La instancia EC2 <code>t3.medium</code> : control total, responsabilidad operativa completa, facturación por hora encendida.
FaaS	<i>Function as a Service</i> . Las funciones Lambda: sin servidor que gestionar, escalado a cero, facturación por milisegundo de ejecución.
DBaaS	<i>Database as a Service</i> . Amazon RDS: motor relacional con parcheo, respaldo y disponibilidad gestionados por el proveedor.
Self-healing	Capacidad del orquestador de reconstruir automáticamente una carga fallida. Distinta de alta disponibilidad, que requiere redundancia de nodos.
Medallion	Patrón de organización del Data Lake en zonas de fidelidad decreciente y valor creciente: cruda, optimizada y de consumo.